Tool Setup and Workflow

Open-Source Analog/Mixed-Signal Integrated Circuit Design Project Initiative

Eli Johnson

October 14, 2026

Abstract

This guide prepares members of the Open-Source Analog/Mixed-Signal Integrated Circuit Design Project Initiative for the first meeting and for every design session after it. Each member works in a personal GitHub repository created from the initiative repository, so that everyone has an independent copy of the tools setup, examples, and their own designs. Section 1 introduces the open-source design flow. Section 2 walks through creating, renaming, and configuring your repository. Section 3 takes you from a clean laptop to a running design environment. Section 4 describes day-to-day work in your repository, including receiving new material after each meeting, and Section 5 explains how groups share a repository during design sessions. No prior circuit design or Git experience is assumed.

Contents

1 Overview **2**
- 1.1 The Design Flow . . . 2
- 1.2 How the Repositories Are Organized . . . 2
- 1.3 How This Guide Fits the Semester . . . 3

2 Creating Your Repository **3**
- 2.1 Prerequisites . . . 3
- 2.2 Sign In and Set Your Git Identity . . . 3
- 2.3 Fork the Initiative Repository . . . 3
- 2.4 Rename or Configure Your Repository Later . . . 4
- 2.5 Clone Your Repository . . . 4

3 Setting Up the Design Environment **4**
- 3.1 Launch the Container . . . 4
- 3.2 Connect to the Linux Desktop . . . 5
- 3.3 Launch Xschem . . . 5
- 3.4 Try the Session 0 Examples . . . 6
- 3.5 Checklist Before Meeting 1 . . . 6

4 Working in Your Repository **6**
4.1 Guiding Principle . . . 6
4.2 Receiving New Meeting Material . . . 6
4.3 Step 1: Synchronize Your Local `main` . . . 7
4.4 Step 2: Create and Publish a Branch . . . 7
4.5 Step 3: Design, Simulate, Commit, and Push . . . 7
4.6 Step 4: Open and Merge a Pull Request . . . 7
4.7 Step 5: Clean Up Locally . . . 8

5 Design Sessions: Group Repositories **8**
5.1 Creating the Group Repository (One Member) . . . 8
5.2 Joining the Group Repository (Everyone Else) . . . 8
5.3 Working in the Group Repository . . . 8

Quick Reference **9**

1 Overview

1.1 The Design Flow

Analog and mixed-signal IC design has traditionally required expensive commercial tools. Open-source process design kits (PDKs) and tools now make it possible to design real, manufacturable circuits on a personal laptop at no cost. Everything we use this semester is free and runs inside a single preconfigured container, so all members work in an identical environment.

Component	Role	Used for
GF180MCU PDK	Process design kit	Transistor models and design rules for the GlobalFoundries 180 nm process.
Xschem	Schematic capture	Drawing circuits and testbenches.
Ngspice	Circuit simulator	Operating-point, DC, AC, and transient analyses; building g_m/I_D lookup tables.
KLayout	Layout editor	Physical layout. Not a focus this semester; used when preparing designs for tape-out.

1.2 How the Repositories Are Organized

Repository	Purpose
Initiative repository	Maintained by the initiative leads (`https://github.com/CU-Open-Source-IC-Design/Analog-Mixed-Signal-Initiative`). Contains the launch scripts, meeting examples, and lookup tables. Members do not edit it directly.
Your repository	Your personal fork of the initiative repository. All of your individual work lives here, and you can pull in new meeting material at any time.
Group repository	Created for each design session by one member of the group and shared with the others (Section 5).

1.3 How This Guide Fits the Semester

Complete Sections 2 and 3 *before* Meeting 1, which opens with tool configuration and moves straight on to MOSFET basics and a common-source amplifier simulated with op, DC, AC, and transient analyses. Arriving with a working environment lets you follow the simulation examples live. Section 5 becomes relevant in the design sessions (Meetings 4 and 6), where members split into groups, each assigned a set of specifications to meet in the time given.

2 Creating Your Repository

2.1 Prerequisites

Install the following applications on your own computer before you begin.

Application	Purpose
GitHub Desktop	Clone your repository and manage branches, commits, and pushes.
Docker Desktop	Run the container holding the PDK and design tools.
TigerVNC Viewer	Connect to the graphical Linux desktop inside the container.

You also need free accounts on GitHub.com and Docker. After installing Docker Desktop, restart your computer and sign in to Docker Desktop.

Windows only: install WSL

The container needs the Windows Subsystem for Linux. Open Command Prompt, run `wsl -install`, and restart. When Ubuntu opens for the first time, create a Linux username and password. macOS users can skip this step.

2.2 Sign In and Set Your Git Identity

1. Open GitHub Desktop and sign in with your GitHub account (**Settings** on Windows, or **GitHub Desktop** → **Settings** on macOS, then **Accounts**).
2. Under **Git** in the same window, confirm that your name and email are filled in. These are attached to every commit you make.

2.3 Fork the Initiative Repository

A *fork* is your own copy of the initiative repository on GitHub. It stays linked to the original, which lets you pull in new material later (Section 4.2).

1. Go to the initiative repository on GitHub.com (`https://github.com/CU-Open-Source-IC-Design/Analog-Mixed-Signal-Initiative`).
2. Click **Fork** (top right), then **Create a new fork**.
3. Set the **Repository name**. Use the format `ams-<yourname>`, for example `ams-jsmith`, so that leads can tell the 20 or so member repositories apart.
4. Optionally add a description, such as *"My designs for the Open-Source AMS IC Design Initiative"*.

5. Leave **Copy the main branch only** checked and click **Create fork**.

Visibility

Forks of a public repository are public. This is in keeping with an open-source initiative, and your repository doubles as a portfolio project.

2.4 Rename or Configure Your Repository Later

All of these are under **Settings** on your repository's GitHub page.

- **Rename: Settings → General → Repository name**, then **Rename**. GitHub redirects the old address, but update GitHub Desktop to match: **Repository → Repository Settings → Remote**, and paste the new URL.
- **Description and topics:** click the gear icon next to **About** on the repository's main page. Topics such as `analog`, `gf180mcu`, and `xschem` make the project easier to find.
- **Collaborators: Settings → Collaborators → Add people**. Needed only for group repositories (Section 5).

Renaming the folder on your computer

Renaming the repository on GitHub does not rename your local folder. To rename it, in GitHub Desktop choose **Repository → Remove** (do *not* move it to the trash), rename the folder in Finder or File Explorer, then choose **File → Add Local Repository** and select the renamed folder. Stop the design container first.

2.5 Clone Your Repository

1. In GitHub Desktop, choose **File → Clone Repository** and select *your* fork (`<your-username>/ams-<yourname>`, not the initiative repository.
2. Keep the default location (`Documents/GitHub/ams-<yourname>`) and click **Clone**. You will open a terminal in this folder in the next section.
3. If GitHub Desktop asks how you plan to use the fork, choose **For my own purposes**. This keeps your branches and pull requests in your own repository.

3 Setting Up the Design Environment

3.1 Launch the Container

1. Open Docker Desktop and leave it running on the **Containers** page. On a first-time setup the list should be empty.
2. Open a terminal (Terminal on macOS, PowerShell or Command Prompt on Windows) and change into your cloned repository folder.
3. Run the launch script for your operating system:

   macOS / Linux:

```
./start_chipathon_vnc.sh
```

Windows:

```
.\start_chipathon_vnc.bat
```

4. Docker will download the container image. The first download takes several minutes; later launches are much faster.

Tip

If the macOS script reports *permission denied*, make it executable once with `chmod +x start_chipathon_vnc.sh` and run it again.

Already have an IIC-OSIC-TOOLS container?

If you used this script for another project (for example, a Chipathon), it will find and reuse that container, which saves to the *other* project's folder. Stop the old container in Docker Desktop, then give this one its own name:

```
CONTAINER_NAME=iic-osic-tools_ams_xvnc ./start_chipathon_vnc.sh
```

To switch projects later, stop one container and start the other with the play button in Docker Desktop.

3.2 Connect to the Linux Desktop

1. In Docker Desktop, confirm that the new container appears and is running. If it is stopped, click the **Start** (play) button.
2. Open TigerVNC Viewer, enter the server address `localhost:5901`, and click **Connect**.
3. When prompted, enter the VNC password `abc123`.

3.3 Launch Xschem

Once connected to the Linux desktop, open a terminal. It starts in the correct environment with the GF180MCU PDK already loaded, so no further setup is required. Launch the schematic editor with:

```
xschem
```

To confirm the PDK is available, press **Shift+I** (insert symbol) in Xschem. The GF180MCU libraries, including devices such as `nfet_03v3` and `pfet_03v3`, should be listed.

From Xschem you can open existing schematics or create new schematics and testbenches. Simulations are run with Ngspice directly from Xschem by clicking **Netlist** and then **Simulate**.

Shared design folder

The directory `/foss/designs/` inside the container is the same folder as `designs/` in the repository you launched the container from. Anything you save there appears in that repository immediately and is visible to GitHub Desktop for committing.

Important

Save all of your work under `/foss/designs/`. Files saved elsewhere in the container are not part of any repository and may be lost when the container is removed.

3.4 Try the Session 0 Examples

The repository includes a `session_0` directory of familiar circuits, such as a voltage divider, an RC low-pass filter, and an LC tank. Each testbench already contains its models and simulation commands, so you can run it right away to confirm that your environment works and to get comfortable with Xschem.

1. In Xschem, choose **File** → **Open** and browse to `/foss/designs/session_0/`.
2. Open any testbench (`.sch`) file.
3. Click **Netlist**, then **Simulate**. Ngspice runs the analyses and opens the resulting plots.

Experimenting

Try changing a resistor or capacitor value and simulating again. If you want to keep your changes, first copy the example into your own folder (for example, `designs/<yourname>/`), as described in Section 4.2.

3.5 Checklist Before Meeting 1

- ☐ GitHub Desktop, Docker Desktop, and TigerVNC are installed.
- ☐ You have forked the initiative repository and named it `ams-<yourname>`.
- ☐ Your fork is cloned and the container image has been downloaded.
- ☐ You can connect to the Linux desktop through TigerVNC.
- ☐ Xschem opens with the GF180MCU libraries available.

If any step fails, bring your laptop to Meeting 1 and we will resolve it during tool configuration.

4 Working in Your Repository

4.1 Guiding Principle

Even in your own repository, do new work on a *feature branch* rather than directly on `main`. Your `main` then always holds working designs, and an experiment that goes wrong can simply be abandoned. This is also the workflow used in the group repositories and in industry.

Sync `main` → Branch → Design & commit → Pull request → Merge & clean up

4.2 Receiving New Meeting Material

New examples and lookup tables are added to the initiative repository as the semester progresses. To bring them into your repository:

1. Go to your repository on GitHub.com. If the initiative repository has changed, a message reports that your branch is some commits behind.
2. Click **Sync fork**, then **Update branch**.
3. In GitHub Desktop, switch to `main` and click **Fetch origin**, then **Pull origin**.

Avoid sync conflicts

Do not edit the provided example files in place. Copy an example into your own subfolder, such as `designs/<yourname>/`, and modify the copy. Likewise, do not edit the `README.md` in your fork. Updates from the initiative repository then never collide with your changes.

4.3 Step 1: Synchronize Your Local `main`

Before creating a branch, sync your fork as above (or at least click **Fetch origin** and **Pull origin** on `main`). Starting from an outdated `main` commonly leads to conflicts later.

4.4 Step 2: Create and Publish a Branch

1. Click the current-branch dropdown, then **New Branch**.
2. Name the branch `feature/<short-description>`, using lowercase letters and hyphens, for example `feature/cs-amp-resistive` (Meeting 1) or `feature/gmid-lut` (Meeting 3).
3. Click **Publish branch** to push the branch to GitHub. This backs up your work from the start.

4.5 Step 3: Design, Simulate, Commit, and Push

Do your design work in Xschem and simulate with Ngspice, saving files normally. When you complete a meaningful unit of work, such as a working schematic, a successful simulation, or a sizing iteration that meets a spec:

1. Return to GitHub Desktop. Your changed files appear in the left pane. Review them to confirm only intended files are included.
2. Write a short, descriptive commit summary (e.g., *"Size input pair with gm/Id = 15; GBW meets spec"*).
3. Click **Commit to feature/<your-branch>**, then **Push origin**.

Commit often. Each commit is a checkpoint you can return to if a change breaks a working design.

4.6 Step 4: Open and Merge a Pull Request

When the work is complete, open a pull request (PR) *within your own repository* to merge the branch into your `main`.

1. On your repository's GitHub page, click **Compare & pull request**.
2. Check that the base repository is *your* fork and the base branch is `main`. GitHub sometimes defaults to the initiative repository; change it if so.

3. Write a short description: what you designed, the specifications targeted and achieved, and how you verified them (analyses run, key results, plots).
4. Click **Create pull request**, then **Merge pull request**, and finally **Delete branch**.

Getting feedback

To ask for feedback on a design, link the PR in the initiative's group chat or add a lead as a collaborator and request their review before merging.

4.7 Step 5: Clean Up Locally

1. In GitHub Desktop, switch to `main`, then click **Fetch origin** and **Pull origin**.
2. Open the branch dropdown, right-click your old branch, and select **Delete**.

5 Design Sessions: Group Repositories

In Meetings 4 and 6, each group designs to its assigned specifications in a shared group repository.

5.1 Creating the Group Repository (One Member)

1. On the initiative repository's GitHub page, click **Use this template** → **Create a new repository**.
2. Name it after the session and group, for example `cs-amp-group2` (Meeting 4) or `ota-group3` (Meeting 6), and click **Create repository**.
3. Under **Settings** → **Collaborators**, click **Add people** and invite each group member by GitHub username.

5.2 Joining the Group Repository (Everyone Else)

1. Accept the invitation from your email or from your GitHub notifications.
2. In GitHub Desktop, choose **File** → **Clone Repository** and clone the group repository.

5.3 Working in the Group Repository

1. In Docker Desktop, stop your personal container. Then open a terminal in the group repository's folder and create a separate container for it, so that `/foss/designs/` points to the group repository:

   ```
   CONTAINER_NAME=iic-osic-tools_group_xvnc ./start_chipathon_vnc.sh
   ```

   Connect with TigerVNC as usual and launch Xschem.
2. Each member works on their own branch, such as `feature/input-pair` or `feature/bias-mirror`, following Steps 1–5 of Section 4.
3. Open pull requests into the group repository's `main` and request a groupmate as reviewer before merging.

Coordinate edits

Decide who is editing which file. Two people changing the same schematic at once will produce a conflict that is difficult to resolve.

When the session ends, stop the group container and start your personal one with the play button in Docker Desktop.

Quick Reference

Item	Value
Process design kit	GF180MCU (GlobalFoundries 180 nm)
Tools	Xschem, Ngspice, KLayout
Your repository name	`ams-<yourname>` (fork of the initiative repository)
Group repository name	`<design>-group<N>` (from template)
Get new material	**Sync fork** on GitHub, then **Pull origin**
Launch script (macOS)	`./start_chipathon_vnc.sh`
Launch script (Windows)	`.\start_chipathon_vnc.bat`
VNC server address	`localhost:5901`
VNC password	`abc123`
Schematic editor	`xschem`
Separate container	`CONTAINER_NAME=<name> ./start_chipathon_vnc.sh`
Shared design folder	`/foss/designs/` ↔ `designs/` in the launched repository
Branch naming	`feature/<short-description>`